

0.41.1. Taille 8 : un carré de 17×17 — de base, ailleurs

Le carré du 0.41, avec une seule lettre, le B, autour d'une deuxième position : (8, 8).

1. Ce qu'il faut comprendre

C'est le carré du 0.41, mêmes réglages (taille 8, sens 1, vitesse 250, 1 tour), avec deux changements qui vont ensemble : **une autre lettre** et **une autre position**.

La lettre : `ALPHABET[1]`, la 2e lettre de l'alphabet, le **B** (on compte à partir de 0 : A = 0, B = 1).

La position : (8, 8) au lieu de (10, 8). Un carré de taille 8 tient entier si son centre est à au moins 8 cases de chaque bord : la colonne entre 8 et $19 - 8 = 11$, la ligne entre 8 et $17 - 8 = 9$. (8, 8) respecte ces limites : le carré de 17×17 tient entier, des colonnes 0 à 16 et des lignes 0 à 16.

C'est la version de base : une seule lettre, un seul carré. Le 0.41.2 mettra les deux carrés ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction carre\(\) — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
carre()         440 o ← main() - une fonction fournie par le langage
main()          38 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 4      52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image() × 4      12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :
carre() x 1 31 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre> // une tuile qui tourne en carré
#include <ALPHABET> // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Le carré du 0.41, ailleurs et avec le B :
    //
    // carre(8, 8, ALPHABET[1], 8, 1, 250, 1);
    //      | | |
    //      | | +-- ALPHABET[1] : la 2e lettre, B
    //      +---+----- le centre (8, 8) : le carré de taille 8
    //                  y tient entier (colonnes 0 à 16, lignes 0 à 16)
    //
    // Les autres réglages sont ceux du 0.41 : taille 8, sens 1, vitesse 250, tours 1.
    carre(8, 8, ALPHABET[1], 8, 1, 250, 1);

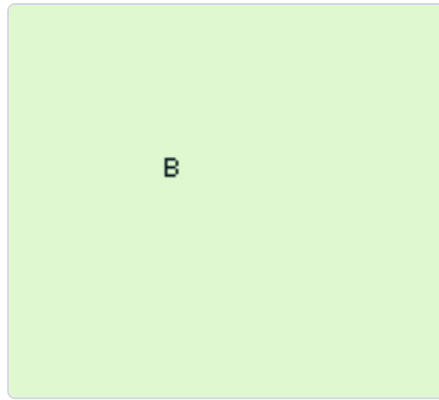
    while (true) {
        image(); // le B est revenu à son centre
    }
}
```

3. Ce qui se passe, étape par étape

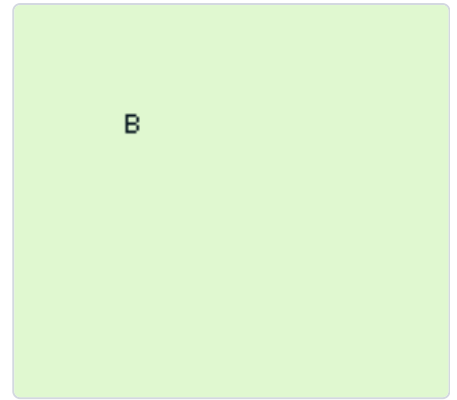
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



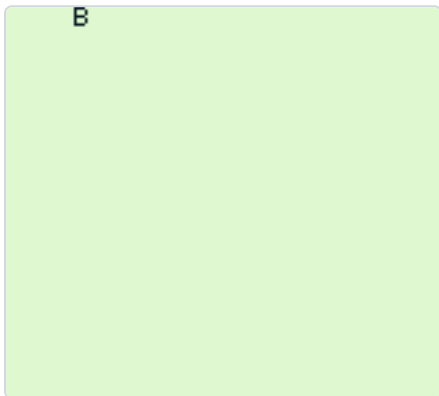
la toute première image — la console vient de démarrer — image 1



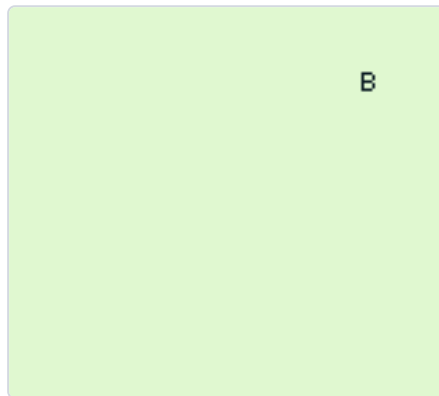
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le même carré qu'au 0.41, mais fait par un B, autour de (8, 8).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B fait le tour autour de (8, 8) : coins (0, 0) et (16, 16)

✓ le B revient à son centre (8, 8) (8,8)

Ensuite : **0.41.2. Taille 8 : un carré de 17 × 17 — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++